

Razvoj web aplikacija - Predavanje 1

Backend arhitektura i skeleton projekta

mag.ing. Josip Torić (SIT UNIZD)

05.10.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Što znači „backend arhitektura” (i što je *loša* arhitektura)
- 2 Kako izgleda tipičan request: frontend → API → DB
- 3 Skeleton projekta: struktura repozitorija i moduli
- 4 Dev okruženje: Docker Compose, env varovi, konfiguracija
- 5 FastAPI: app factory, routeri, dependencies, health endpoint
- 6 Git workflow: commitovi, tagovi i `git diff` kao alat za učenje

Na kraju predavanja imate **pokrenut** projekt: baza + API + dokumentiran način pokretanja.

Ishodi učenja (što morate znati objasniti)

Nakon ovog predavanja student može:

- objasniti tok HTTP zahtjeva kroz slojeve backend aplikacije
- razlikovati **router**, **service** i **DB layer**
- postaviti razvojno okruženje: Docker Compose + env varovi
- objasniti zašto koristimo **app factory** i modularnu strukturu
- koristiti `git diff` za razumijevanje promjena u kodu

Zašto se uopće bavimo arhitekturom?

- Jer „radi mi” nije isto što i „održivo je”
- Jer na obrani ispitujem **razumijevanje**, ne copy/paste
- Jer se problemi pojavljuju kad projekt naraste:
 - više endpointa, više pravila, više korisnika
 - više developera (vi u timu)
 - više bugova i edge caseova

Arhitektura = kako je sustav organiziran i gdje živi odgovornost.

Najčešći anti-pattern: „Sve u routeru”

Sve u routeru

- jedna velika datoteka: HTTP + validacija + DB upiti + poslovna pravila + errori
- ne možeš testirati bez HTTP-a
- svaka promjena razbije nešto drugo

Slojevi

- router je tanak: samo HTTP
- poslovna logika u service-u
- DB detalji u repository/DB layeru

Na obrani: tko ima sve u routeru, ne zna objasniti *što je gdje* i *zašto*.

Request flow (mentalni model)

- 1 Frontend šalje HTTP request (npr. POST /auth/login)
- 2 Router parsira request + provjerava auth dependency
- 3 Service provodi poslovna pravila (npr. rokovi, ownership)
- 4 Repository/DB layer radi upit (transakcija)
- 5 Service vraća rezultat (ili baca domain error)
- 6 Router mapira rezultat u HTTP response (status code + JSON)

HTTP (router) je samo **transport**. Business pravila su **domena**.

REST u praksi (minimum koji tražim)

Resource-oriented URL-ovi: GET /lifters, POST /lifters, GET /lifters/{id}, PATCH /lifters/{id}, DELETE /lifters/{id}.

Status codeovi nisu dekoracija:

Kod	Kada
200 / 201	uspjeh / resurs kreiran
400 / 422	neispravan zahtjev / validacija inputa pala
401 / 403	nisi prijavljen / nemaš ovlasti
404	resurs ne postoji
409	konflikt, npr. duplikat prijave
500	greška servera

GET, PUT i DELETE su u pravilu idempotentni.

DTO i validacija (zašto Pydantic)

Backend mora validirati input - frontend se ne računa kao zaštita. DTO, odnosno schema, je ugovor API-ja: definira što točno ulazi i što izlazi.

Pydantic taj ugovor provodi automatski: nevaljan input dobije jasan 422 s opisom greške, a iz istih schema se generiraju OpenAPI modeli.

Na obrani pitam

Gdje je schema, što validira, koji error vraća i zašto.

Konfiguracija: 12-factor i env varovi

- Konfiguracija ne smije biti hardkodirana
- Tajne ne idu u git:
 - DB password, JWT secret, API base URL
- `.env` je za lokalno, `.env.example` je za repo
- U produkciji: env varovi dolaze iz platforme (Railway/Render)

Ako promijeniš okruženje (lokalno → produkcija), ne smiješ mijenjati kod.

Docker Compose: zašto ga koristimo?

Standardizira PostgreSQL za sve studente, uklanja „meni ne radi na Windowsu/Macu” i omogućava brzo resetiranje okruženja.

U Composeu	Lokalno
db - Postgres + volume	backend - uvicorn --reload frontend - Vite dev server

Backend i frontend ne dockeriziramo radi brzine razvoja.

Repo struktura (monorepo)

```
repo/  
  api/ # FastAPI backend  
  web/ # Vue frontend  
  docker-compose.yml  
  .gitignore  
  README.md  
  .env.example
```

Jedan repo = lakša integracija, jedan README, jedna priča na obrani.

Backend struktura (minimalno, ali ozbiljno)

```
api/  
  app/  
    main.py # app factory + include routers  
  core/  
    config.py # env config  
    deps.py # dependencies (auth, db)  
    errors.py # custom errors / handlers  
    logging.py # logging config  
  routers/  
    health.py  
  schemas/  
  models/  
  services/  
  repositories/  
  tests/  
  pyproject.toml (ili requirements.txt)  
  alembic/ (kasnije)
```

App factory: zašto?

- Omogućava:
 - testiranje bez pokretanja servera
 - konfiguraciju ovisno o okruženju
 - modularno dodavanje routera/middlewarea
- Smanjuje „global state” probleme

`create_app()` napravi aplikaciju. `uvicorn` samo **servira**.

Primjer: minimalni main.py

```
from fastapi import FastAPI
from app.routers.health import router as health_router

def create_app() -> FastAPI:
    app = FastAPI(title="Powerlifting Registrations API")
    app.include_router(health_router, prefix="/health", tags=["health"])
    return app

app = create_app()
```

Health endpoint (zašto je bitan?)

- Najjednostavniji endpoint koji potvrđuje:
 - da se aplikacija diže
 - da routing radi
 - (kasnije) da DB konekcija radi
- U produkciji se često koristi kao readiness/liveness check

GET /health → { "status": "ok" }.

Primjer: routers/health.py

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/")
def health():
    return {"status": "ok"}
```

Konfiguracija: core/config.py

- Čitamo env varove na jednom mjestu
- Ne razbacujemo `os.getenv` po kodu

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    ENV: str = "dev"
    DATABASE_URL: str
    JWT_SECRET: str = "change-me"
    JWT_ISSUER: str = "sit-unizd"

settings = Settings()
```

Pydantic settings je najčišći put za konfiguraciju.

Logging (što logiramo, što ne?)

Logiramo	Ne logiramo
request metadata: ruta, status code	lozinke, tokene, osobne identifikatore
error stack trace u dev okruženju	kompletan request body ako sadrži osjetljive podatke
ključne domenske događaje, npr. „registration withdrawn”	

Log nije baza. Log je trag za debugging.

Logging: minimalna konfiguracija

```
# app/core/logging.py
import logging

def setup_logging(debug: bool = False) -> None:
    logging.basicConfig(
        level=logging.DEBUG if debug else logging.INFO,
        format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    )

# u servisu:
logger = logging.getLogger(__name__)
logger.info("club created: id=%s", club.id)
```

Logger se dohvaća po modulu preko `__name__` pa svaki zapis kaže odakle dolazi. U produkciji sve ide na stdout, a platforma skuplja logove - detalji u Predavanju 12.

Error handling (na razini arhitekture)

- Dosljedan format errora:
 - code, message, (opc.) fields
- Razdvajamo:
 - **domain error** (poslovno pravilo) vs **HTTP error**
- Kasnije: global exception handler mapira domain error u status code

Ako to ne dogovorite odmah, poslije je kaos.

Tooling: lint i format (zašto Ruff?)

- **Ruff** je danas i linter i formatter, jedan alat za oboje:
 - `ruff check .` - lint: importi, greške, stil
 - `ruff format .` - formatiranje, jedan stil bez rasprave
- Formatter je naslijedio Black i drži isti stil koda
- Smanjuje code review overhead i olakšava timski rad

Ako u repou vidim nered, na obrani se prvo bavimo time i gubite vrijeme.

Testiranje: što znači „testabilna arhitektura“?

- Ako je sva logika u routeru, testiraš sve preko HTTP-a $\Rightarrow$ sporo i krhko
- Ako je logika u service-u, možeš testirati:
 - pravila (rokovi, ownership) bez HTTP sloja
- FastAPI omogućava i integration testove (TestClient/httpx)

Danas postavljamo skeleton tako da se testovi kasnije pišu bez muke.

Git kao nastavna metoda: commitovi i diff

Svaki commit je mala lekcija.

Veliki commit = loša lekcija.

Ne live-codeam: pokazujem unaprijed pripremljen projekt kroz commitove. Vi učite gledajući **što se promijenilo** i **zašto** - git diff je alat za razumijevanje, ne samo za rješavanje konflikata.

Kako čitati diff (brza pravila)

- Prvo razumij **što** je cilj promjene (commit message)
- Onda gledaj:
 - novi fileovi vs izmjene postojećih
 - javne API promjene (npr. novi endpoint)
 - promjene u configu (env, compose)
- Tek na kraju detalji implementacije

Korisne git naredbe

```
# pregled povijesti  
git log --oneline --decorate  
  
# diff izmedu commitova  
git diff <commit1>..<commit2>  
  
# diff u konkretnom commitu  
git show <commit>  
  
# promjena na konkretan tag  
git checkout lecture-01-end
```

Commitovi za Predavanje 1 (što obrađujemo danas)

- chore: init monorepo (api, web) with tooling configs
- chore: add docker-compose postgres and env example files
- feat(api): bootstrap FastAPI app with health check and version
- refactor(api): add app factory and basic module structure
- docs: add local dev guide and project conventions

Na satu: prolazimo diffeve i objašnjavamo *zašto* je to u toj datoteci.

Demo: docker-compose.yml (minimalno)

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: pl_reg
      POSTGRES_USER: pl_user
      POSTGRES_PASSWORD: pl_pass
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Lozinke su demo - u praksi se stavljaju u .env.

Pokretanje baze i API-ja (danas)

```
# baza
docker compose up -d db

# backend
cd api
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
uvicorn app.main:app --reload
```

uv objedinjuje venv i pip, brži je i danas je standard za nove projekte - requirements.txt ostaje isti.

Checkpoint: otvorite `http://127.0.0.1:8000/health` i dobijte "ok".

Swagger / OpenAPI (zašto je bitno od prvog dana?)

- API dokumentacija se generira automatski
- Olakšava testiranje endpointa
- Frontend se oslanja na ugovor API-ja

`/docs` i `/openapi.json` su vaši prijatelji.

Što ide sljedeće (Predavanje 2)

- Modeliranje podataka: Club i User
- SQLAlchemy engine + session
- Alembic migracije (inicijalno)
- Seed admina i demo kluba

Pitanja?